TREASURE HUNT

ROMAN PLAKHOTNIUK, UI PROJECT 2

TASK TARGET

A treasure hunter walks on a grid and has to collect every treasure. The walk is not searched for directly: an evolutionary algorithm evolves programs for a small virtual machine, and each program's output is the sequence of moves.

QUICK START

Requires **UV**.

Create .venv and install dependencies:

```
uv sync
```

Solve grid.txt with default settings:

```
uv run project2.py
```

Make graphs from every logged run:

```
uv run plot.py runs.csv
```

project2.py uses only the standard library, so *python3 project2.py* works without uv as well. Only *plot.py* needs *pandas* and *matplotlib*.

PROJECT STRUCTURE

File	Purpose
project2.py	Grid, virtual machine, evolutionary algorithm, CLI.
plot.py	Graphs and summary table from the logged runs.
experiment.py	Runs the best configuration next to the default on new seeds.
compare.py	Compares every method against the default settings.
grid.txt	The default map from the assignment (7x7, 5 treasures)
experiments/	Logs and summary tables written per generation by *compare.py* and *experiment.py*
plots/	Graphs

pyproject.toml uv.lock	Project metadata and pinned dependencies

HOW IT WORKS

GRID

The map is a plain text file. Each character is one cell:

Character	Meaning
S	Start – exactly one
T	Treasure – at least one
anything else	Empty cell

- The width is the longest line and empty lines at the end are ignored.
- The hunter moves H (up), D (down), P (right) and L (left).
- A treasure counts as found only when the hunter stands on its cell.

VIRTUAL MACHINE

The machine has *64 memory cells of 1 byte*. Instructions and data share the same memory, so programs can modify themselves. Every byte is a valid instruction:

Instruction	Encoding	Effect
INC a	*00aaaaaa*	*memory[a] += 1* (wraps 255 → 0)
DEC a	*01aaaaaa*	*memory[a] -= 1* (wraps 0 → 255)
JMP a	*10aaaaaa*	Continue at address *a*
WRT a	*11aaaaaa*	Output a move from the *last 2 bits of memory[a]*

WRT maps the last 2 bits of the cell to a direction:

Last 2 bits	*00*	*01*	*10*	*11*
Move	H ↑	D↓	P→	L←

INC and *DEC* change the whole byte. Incrementing *00111111* gives *01000000*, so an *INC* can turn into a *DEC*.

Execution starts at address 0. A program stops when any of these happens:

1. the hunter has found all treasures,
2. the hunter steps off the grid,
3. 500 instructions have been executed,
4. execution runs past the last cell (address 63) – unless --wrap is set, which continues from address 0 instead.

FITNESS

$$\text{Fitness} = \text{Treasures found} + 1 - 0.001 \cdot \text{Moves}$$

Treasures always matter most. Among programs that find the same number, the shorter path wins. For example *5.960* means all 5 treasures in 40 moves. Fitness lies between 0.5 and about 6.

A program that never moves gets the lowest fitness: *0.5*. Otherwise doing nothing (fitness 1.0) would beat a program that explores but finds nothing yet (e.g. 0.999).

EVOLUTIONARY ALGORITHM

An *individual* is a list of 64 bytes – the initial memory of the machine.
Every evaluation copies the individual into the machine, so the self-modifying run never changes the original genes.

1. INITIALIZATION
   --pop-size individuals. The first *--init-cells* cells are random bytes (0–255), the rest are zero.

2. EVALUATION
   Every individual is run on the machine and gets a fitness.

3. STOP CHECK
   If the best individual found all treasures, its program and path are printed and the run ends.
   (with *--find-best* the run continues to shorten the path). After *--generations* generations the best solution so far is printed and the user decides whether to run another batch.

4. ELITISM
   The *--elite* best individuals are copied into the next generation unchanged.

5. SELECTION OF TWO PARENTS
   a. *Roulette* – the chance to be picked is proportional to fitness.
   b. *Tournament* – pick *--tournament-size* random individuals, the best one becomes a parent. Repeated for each parent.

6. CROSSOVER
   a. *One-point* – one random cut; the children swap the tails.
   b. *Two-point* – two random cuts; the children swap the middle part.

7. MUTATION

All possible mutations:

Mutation	Applied	Default Probability
Bit flip – flip one random bit	Cell	0.02
Random cell – replace with a random byte	Cell	0.01
Invert – flip all 8 bits	Cell	0.005
Swap – exchange two neighbouring cells	Individual	0.05
Insert – insert a random byte, drop the last cell	Individual	0.05
Delete – remove a cell, append a zero	Individual	0.05

Heavier per-cell mutations are less likely. An individual always stays 64 cells long.

8. LOOP

Steps 5–7 repeat until the new generation is full, then the algorithm continues with step 2.

USAGE

Main command:

```
uv run project2.py [options]
```

Show all options with their defaults:

```
uv run project2.py --help
```

RUN OPTIONS

Option	Default	Description
--grid FILE	*grid.txt*	Map to solve
--seed N	Random	Random seed; the same seed and settings give the same run
--find-best	Off	Don't stop at the first solution, keep shortening the path
--delay SEC	0.15	Seconds between animation frames (0 = no waiting)
--export FILE	*runs.csv*	CSV file the run is appended to

EVOLUTION OPTIONS

Option	Default	Description
--pop-size N	200	Individuals per generation (at least 20)
--generations N	500	Generations before asking whether to continue
--elite N	2	Best individuals copied unchanged (0 = no elitism)
--selection STRATEGY	*roulette*	Roulette or tournament
--tournament-size N	3	Individuals per tournament
--crossover STRATEGY	*one_point*	One_point or two_point
--init-cells N	64	Cells randomized in the first generation (1–64)
--wrap	off	Continue from address 0 after the last cell instead of halting

MUTATION OPTIONS

Probabilities (0–1): *--bit-flip*, *--random-cell*, *--invert*, *--swap*, *--insert*, *--delete*

Tournament of 2, two-point crossover, no elitism:

```
uv run project2.py --selection tournament --tournament-size 2 --crossover two_point --elite 0
```

Only the first 16 cells random, no insert/delete mutations:

```
uv run project2.py --init-cells 16 --insert 0 --delete 0
```

Keep improving the path for 1000 generations:

```
uv run project2.py --find-best --generations 1000
```

LOGGING RUNS

Every run appends one row per generation to *runs.csv* (or the file given by *--export*).
A header is written only when the file is new, so many runs can share one file.

Columns	Content
grid, seed, pop_size, elite, selection, tournament_size, crossover, init_cells, wrap, bit_flip … delete	Settings of the run – the same in every row of a run.
generation	Generation number.
best_fitness, avg_fitness, worst_fitness	Fitness of the best individual, population average, worst individual.
best_so_far	Best fitness of the whole run so far.
treasures, moves	Treasures found and moves made by the best individual of the generation.
unique	Number of distinct individuals in the population.
elapsed	Seconds since the run started.

Because the settings are repeated in every row, runs can be grouped by any setting later – no separate files or naming conventions are needed.

EXPERIMENTS AND PLOTS

RUNNING MANY SEEDS

One run says little – results vary a lot between seeds. Two scripts run many seeds in parallel on all CPU cores.
Every run calls *project2.main* silently – the same code as the command line – and is logged like any other run.

Script	What it runs	Results
compare.py	every method against the default settings, seeds 1–100	Comparison of methods
experiment.py	the best configuration next to the default settings, new seeds 1001–1100	Best configuration

100 seeds per configuration:

```
uv run experiment.py
```

Quicker:

```
uv run experiment.py --seeds 30
```

experiment.py writes the logs to *experiments/experiment/,* all runs together to *experiments/experiment.csv*, the summary table to *experiments/experiment.txt* and the graphs to *plots/*.

For a one-off comparison without editing the scripts, loop over *project2.py*; *< /dev/null* answers "no" to the continue prompt:

```
for cx in one_point two_point; do
  for s in $(seq 1 30); do
    uv run project2.py --seed $s --crossover $cx --generations 300 --
delay 0 --export crossover.csv < /dev/null > /dev/null
  done
done
uv run plot.py crossover.csv --group crossover --out plots/crossover
```

PLOTS

100 seeds per configuration:

```
uv run plot.py runs.csv
```

Compare by any setting column(s):

```
uv run plot.py runs.csv --group selection crossover
```

Setting output file and showing plots in the window:

```
uv run plot.py runs.csv --out plots/experiment1 --show
```

Option	Default	Description
csv	-	CSV written by *project2.py*
--group COL	*selection*	Setting column(s) that define the groups to compare
--out DIR	*plots*	Output directory
--show	off	Also open the graphs in a window

The grid file named in the CSV has to be next to *plot.py* (it is used to count the treasures).

COMPARISON OF METHODS

compare.py compares every method against the default settings (roulette selection, one-point crossover, elite 2, all mutations at their default rates, 64 random cells, halt at the end of memory).
Each comparison changes exactly one thing. Every configuration runs over the same 100 seeds (1–100) with at most 300 generations, so the differences come from the method and not from luck. Each run calls *project2.main* in a worker process – the same code as the command line – and the runs are spread over all CPU cores:

All comparisons, 100 seeds per configuration:

```
uv run compare.py
```

Quicker:

```
uv run compare.py --seeds 30
```

Only some comparisons:

```
uv run compare.py elitism tournament_size
```

The configurations and comparisons are defined in CONFIGURATIONS and COMPARISONS at the top of *compare.py*; a new comparison is one entry in each.

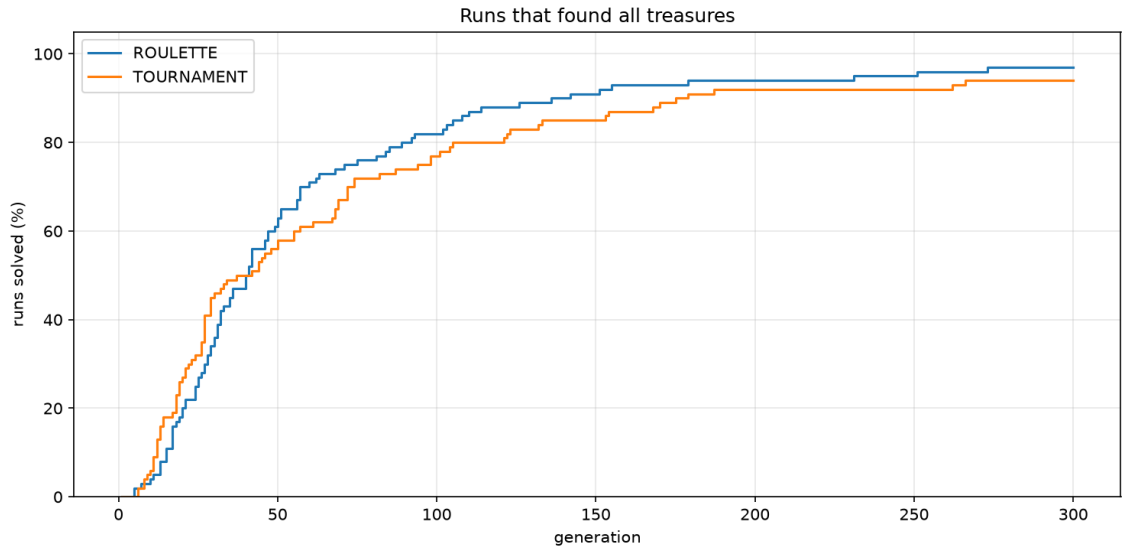
Summary tables are in *experiments/<comparison>.txt*, all graphs (fitness, success, results) in *plots/<comparison>/*

OVERVIEW

Comparison	Configuration	Solved	Solved by gen. 50	Gens median	Gens mean	Moves median
–	**default** (roulette, one-point, elite 2)	97 %	63 %	40	55.3	44
Selection	tournament (size 3)	94 %	58 %	32.5	57.6	38
Tournament size	2	72 %	43 %	37.5	80.9	44.5
	3	94 %	58 %	32.5	57.6	38
	5	**98 %**	**74 %**	**21**	**39.8**	**35**
Crossover	two-point	98 %	68 %	37	51.8	46
Elitism	0	82 %	45 %	47.5	68.1	57
	10	**100 %**	**81 %**	**27.5**	**38.1**	46
Mutation	no swap / insert / delete	**100 %**	**76 %**	**29**	**36.7**	47
	double per-cell rates	99 %	54 %	47	57.7	50
Initialization	16 random cells	98 %	39 %	54	71.7	44.5
End of memory	wrap	99 %	70 %	31	45.4	48
Combined	tournament 5, elite 10, no swap / insert / delete, wrap	98 %	**89 %**	**17**	**25.3**	44

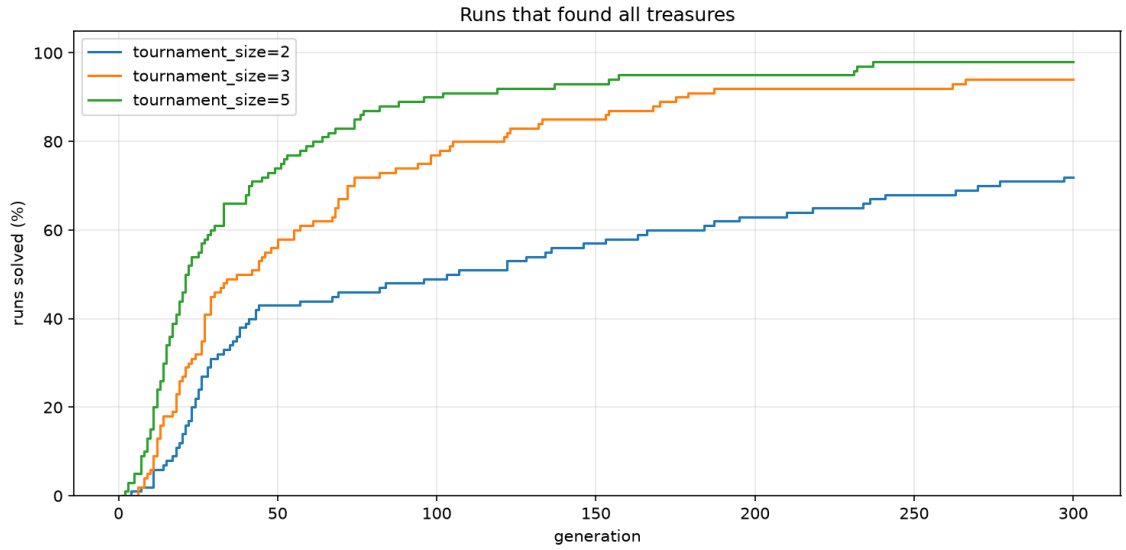
SOLVED is the share of the 100 runs that found all treasures within 300 generations; *gens* and *moves* count only solved runs. With 100 runs per configuration, differences of 2–3 runs (e.g. 97% vs 98 %) and a few generations in the median are within random variation – only larger gaps say something about the method.

SELECTION



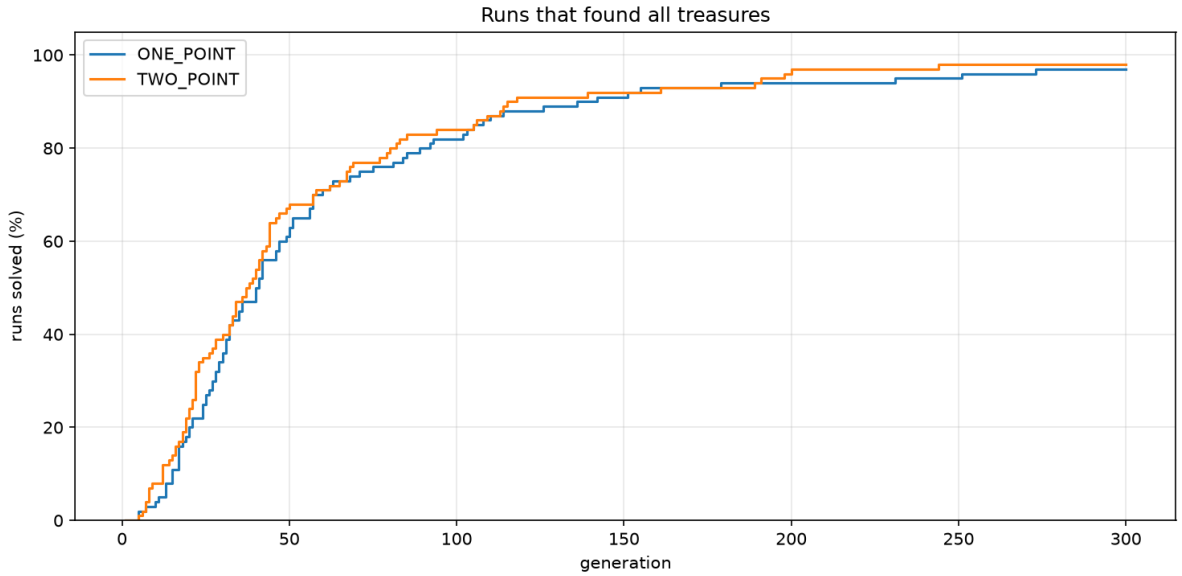
Roulette solved 97 % of the runs, tournament (size 3) 94 %; tournament is faster at the start (median 32.5 vs 40 generations) and finds shorter paths (38 vs 44 moves).
Too much selection pressure is not the reason – the tournament-size comparison shows that a tournament of 5 does better than roulette; a tournament of 3 is simply too weak.

TOURNAMENT SIZE



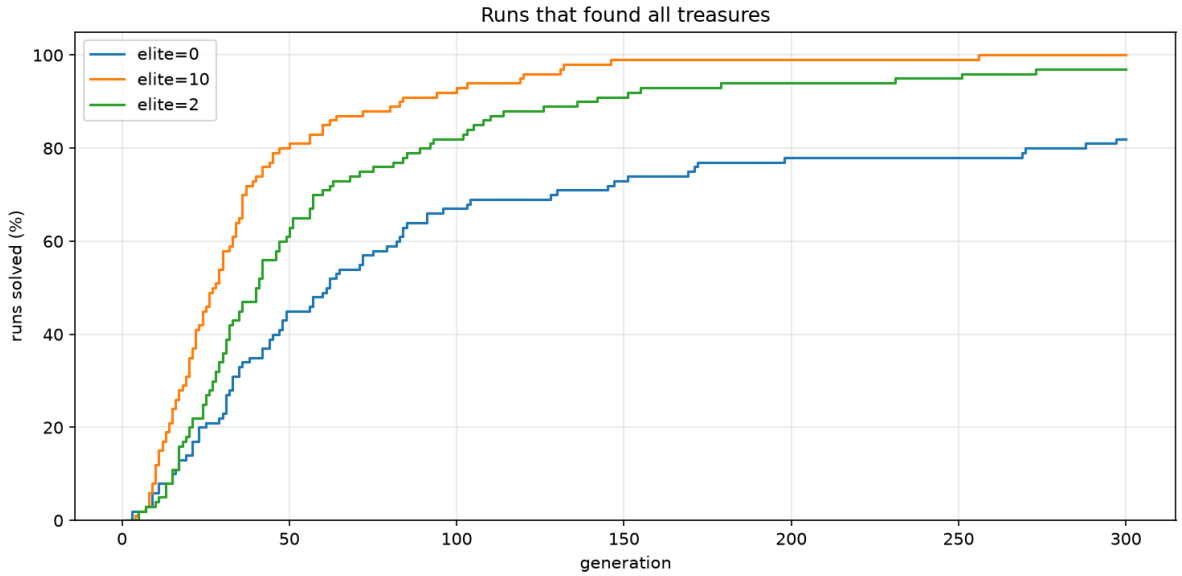
The clearest effect of all comparisons: *the larger the tournament, the better on every measure*. Size 5 solved 98 % of the runs with a median of 21 generations, size 3 94 % (32.5), size 2 only 72 % (37.5), and the size-2 curve keeps rising slowly until the end. A larger tournament makes it more likely that the best programs become parents. With a tournament of 2, a parent only has to beat one random opponent, so average programs become parents almost as often as good ones and improvements spread too slowly.

CROSSOVER



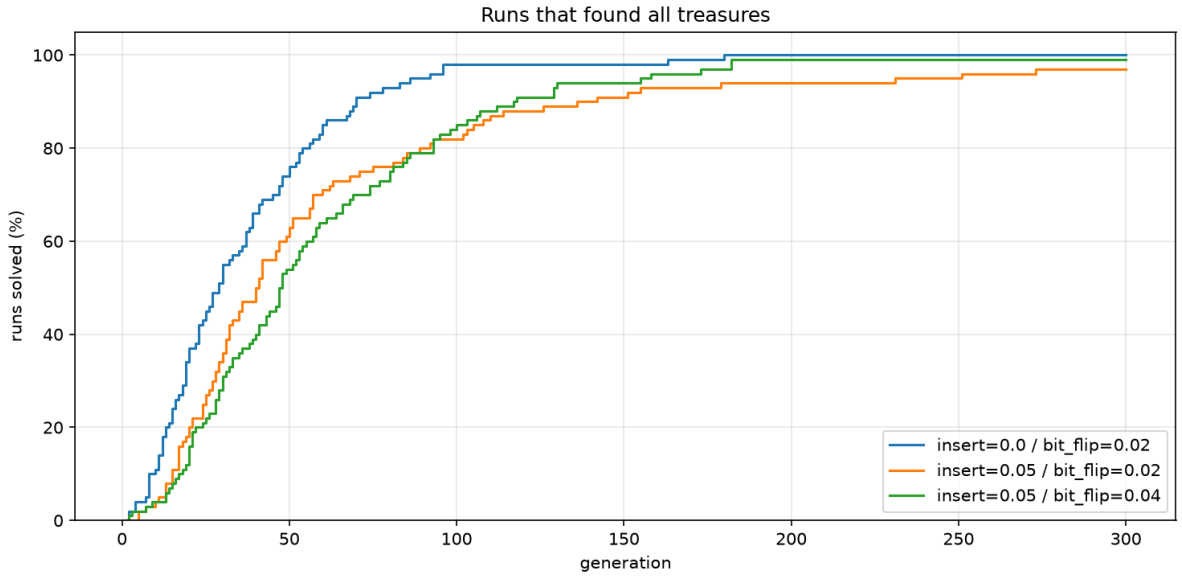
No real difference: 97 % vs 98 % solved, median 40 vs 37 generations, 44 vs 46 moves – all within random variation. Both operators keep long blocks of cells together, so loops and the jumps inside them usually survive. The cut positions are random in both cases, which makes the two operators behave very similarly on 64-cell programs.

ELITISM



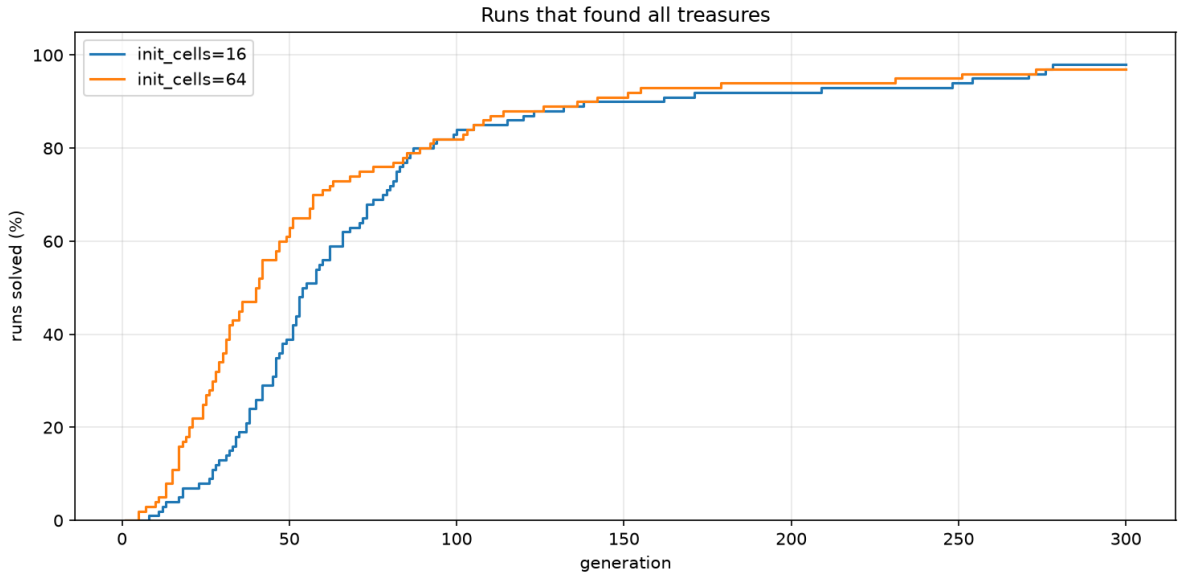
Elitism has a large effect. Without it only 82 % of the runs succeed and the solutions are the longest (57 moves). Crossover and mutation almost always damage a good program, so without elitism the best program found so far can simply disappear from the population, and the run has to find it again. Keeping 2 programs is already enough for 97 %; keeping 10 gives 100 % and a median of 27.5 generations – several good, different programs survive, and crossover can combine them.

MUTATION



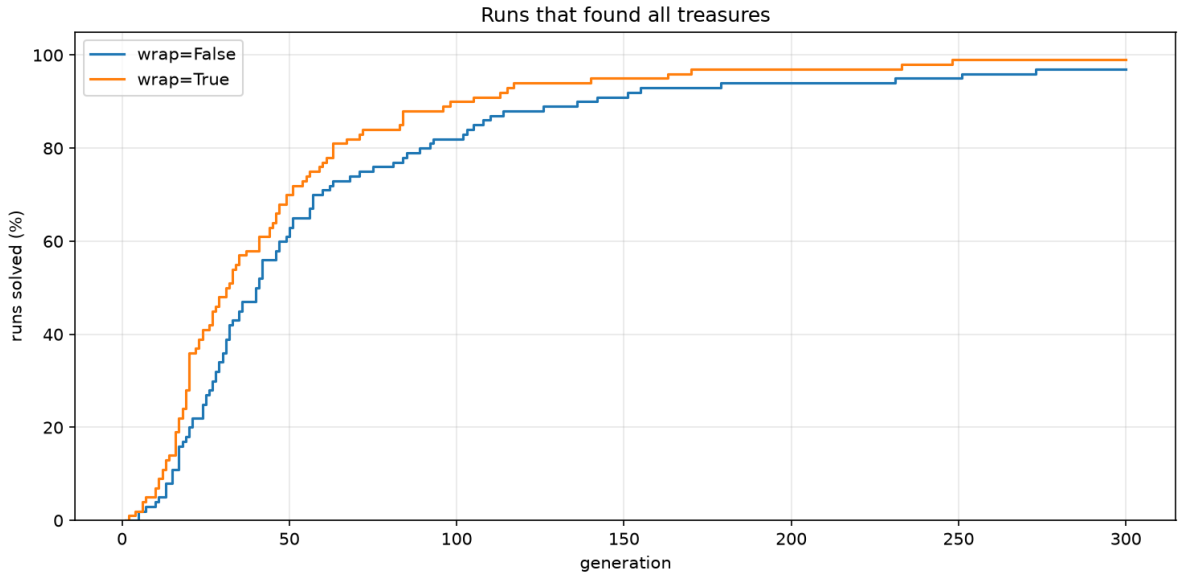
- *Without the structural mutations* (swap, insert, delete) every run succeeded and the median dropped from 40 to 29 generations. Insert and delete shift all the following cells by one, which breaks every jump target and address in that part of the program – most such children are much worse than their parents. Per-cell mutations change only one cell and are enough to explore.
- *Doubling the per-cell rates* (bit flip 0.04, random cell 0.02, invert 0.01) keeps the success rate (99 %) but slows the search (median 47) and gives longer paths (50 moves): more children are damaged, and the population average at generation 50 falls to 1.76 from 2.05.

INITIALIZATION



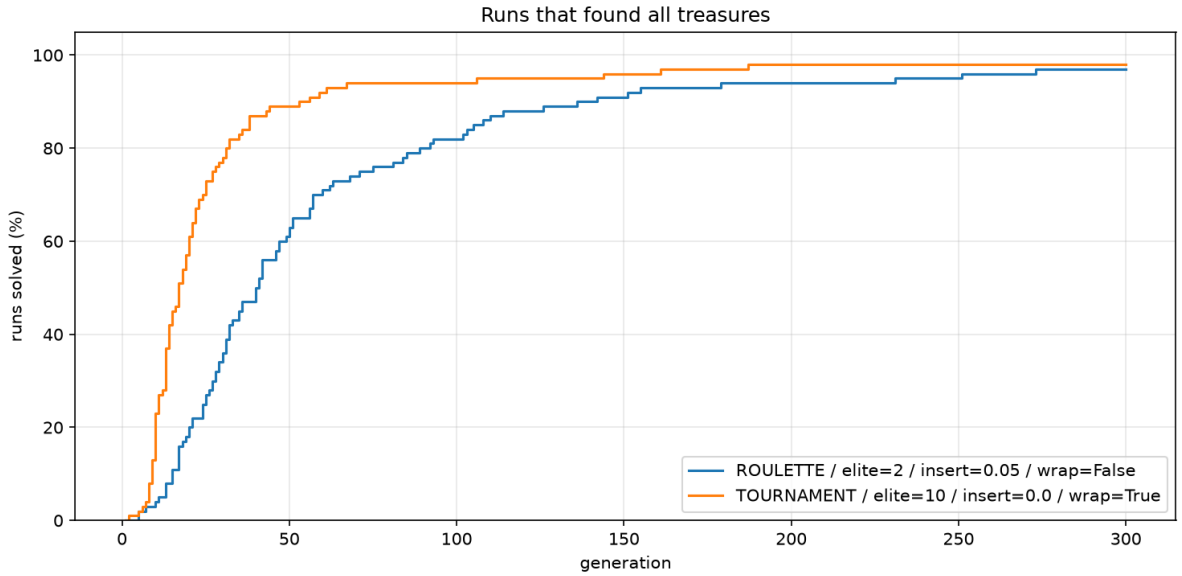
With only the first 16 cells random, *the start is clearly slower* – 8 % of the runs solved by generation 25 vs 27 %, median 54 vs 40 generations. The remaining 48 cells are zero, and 00000000 is the instruction INC 0, so the end of every program is a long block of useless increments; the first generation contains fewer working programs. The final success rate is the same (98 % vs 97 %): mutation fills the empty cells over time.

END OF MEMORY



Letting the program continue from address 0 after the last cell helps: 99 % solved, median 31 vs 40 generations. With wrap-around every program is a loop, so a short sequence of writes can produce a long walk without evolving a working JMP first. The paths are slightly longer (48 vs 44 moves), probably because looping programs tend to repeat the same moves.

COMBINED



The best variant of each comparison together – *tournament of 5, elite 10, no swap / insert / delete, wrap* – is by far the fastest configuration: median 17 generations instead of 40, 73 % of the runs solved by generation 25 instead of 27 %, 89 % by generation 50 instead of 63 %.** The final success rate stays at the default level (98 % vs 97 %; two runs were still unsolved after 300 generations), so the combination mainly makes the search faster rather than more reliable. On new seeds it solved all 100 runs.

CONCLUSIONS

1. ### SELECTION PRESSURE AND ELITISM MATTER MOST
   Larger tournaments and more elites make the search faster and more reliable; without elitism, good programs get lost.

2. ### MUTATIONS SHOULD BE SMALL
   Changing single cells works; inserting or deleting cells shifts addresses and damages programs, and higher rates slow the search down.

3. ### THE PROGRAM REPRESENTATION MATTERS
   Wrap-around and fully random initial programs make working programs easier to find.

4. ### THE CROSSOVER TYPE DOES NOT MATTER FOR THIS PROBLEM

BEST CONFIGURATION

The comparison of methods picked the best variant of every setting. Together they give:

```
uv run project2.py --selection tournament --tournament-size 5 --elite 10 --swap 0 --insert 0 --delete 0 –wrap
```

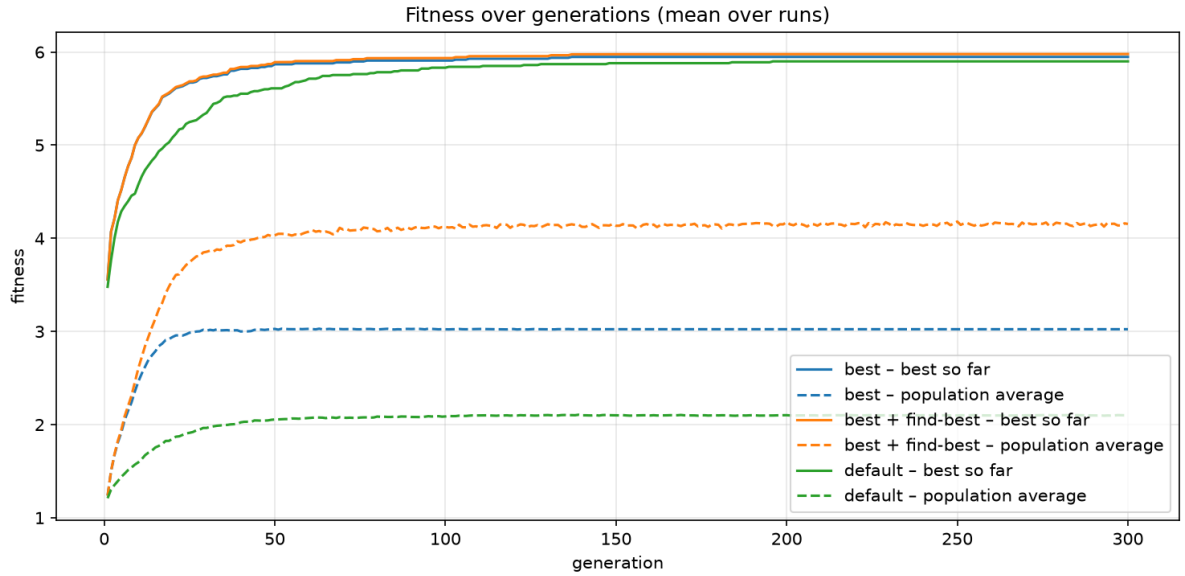
These settings were chosen on seeds 1–100, so their result on exactly those seeds may be a little optimistic. *experiment.py* therefore runs them again on 100 new seeds (1001–1100), next to the default settings, and once more with *--find-best* to see how short the paths get.

Configuration	Solved	Solved by gen. 25	Solved by gen. 50	Gens median	Gens mean	Score	Moves median
best	**100 %**	**74 %**	**92 %**	**16**	**24.0**	**24.0**	**40**
best + find-best	**100 %**	**74 %**	**92 %**	**16**	**24.0**	**24.0**	**18**
default	**96 %**	**34 %**	**66 %**	**32**	**43.9**	**54.2**	**44**

Gens and *moves* count only solved runs. The *score* combines speed and reliability: it is the mean number of generations to find all treasures, where an unsolved run counts as 300 (the generation limit). Lower is better.

On the new seeds the best configuration does even better than on seeds 1–100, where 2 of its runs stayed unsolved: every run found all treasures, half of them within 16 generations, and its score is less than half of the default's. The choice holds on seeds it has never seen.

FITNESS OVER GENERATIONS



Each line is a mean over all runs of a group. The solid line is the best fitness found so far, the dashed line the average fitness of the whole population. The whole-number part is roughly treasures found + 1, so 6 means all treasures; a mean between two whole numbers means the runs disagree (5.5 ≈ half of the runs have all treasures). Runs that finished early keep their last value until the end of the graph.

What it shows:

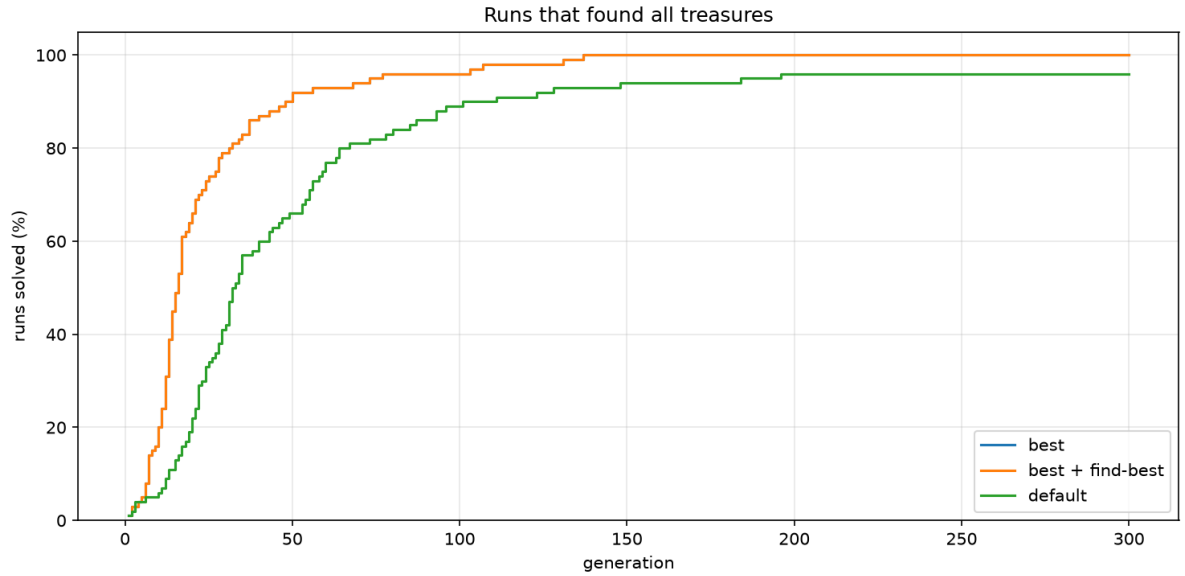
- THE BEST CONFIGURATION CLIMBS FASTER
  Already the random first generation finds 2–3 treasures (mean best fitness about 3.5 in all groups). By generation 10 the best configuration is at 5.08 against 4.58 for the default, by generation 25 at 5.67 against 5.25 – most of its runs have all treasures by then.

- ITS POPULATION AVERAGE IS HIGHER
  About 3.0 against 2.1. A tournament of 5 and 10 elites push the population towards the best programs, and without insert/delete mutations fewer children are destroyed.

- THE FLAT DASHED LINES ARE RUNS THAT STOPPED
  Without *--find-best* a run ends at its first solution, and its population average stays frozen at that moment. With *--find-best* the run goes on: the population fills up with programs that find every treasure, and its average keeps rising to about 4.15.

- THE AVERAGES STAY FAR BELOW THE BEST
  Most children of a good program are much worse – changing one byte can break a jump or a loop – so progress comes from the best individuals, not from the population as a whole.

RUNS THAT FOUND ALL TREASURES



The share of runs that had found all treasures by each generation; every step up is one run reaching a solution. The higher and further left the curve, the better. Where a curve crosses 50% is the generation by which half of the runs are solved; its final height is the success rate within the generation limit.

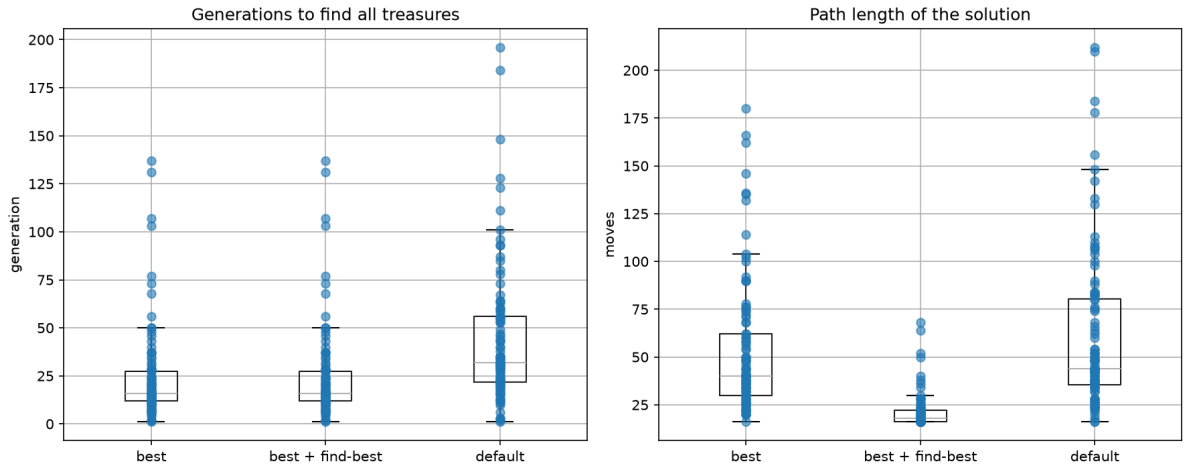
What it shows:

- THE "BEST" LINE IS HIDDEN UNDER "BEST + FIND-BEST"
  Both use the same seeds, and the runs are identical until all treasures are found – `--find-best` only changes what happens after that.

- THE BEST CONFIGURATION SOLVES RUNS ABOUT TWICE AS FAST
  half of the runs by generation 16 against 32, 74 % against 34 % by generation 25, 92 % against 66 % by generation 50.

- ALL 100 OF ITS RUNS WERE SOLVED BY GENERATION 137
  The default solved its last run in generation 196 and left 4 runs unsolved after 300 generations.

SOLVED RUNS



Only runs that found all treasures.
Left: the generation in which each run found them.
Right: the number of moves of that solution.
Every dot is one run; the box holds the middle half of the runs, the line inside it is the median and the whiskers show the usual range. A lower box is better, a narrower box means more predictable behaviour.

What it shows:

- FEWER GENERATIONS AND LESS SPREAD
  The middle half of the best configuration's runs needed 12–27 generations, the default's 22–56.

- THE FIRST SOLUTION IS STILL LONG
  median 40 moves against 44 for the default. The configuration makes the search faster, but a run stops at the first path that collects every treasure, however long it is.

- --FIND-BEST BRINGS THE PATHS CLOSE TO THE OPTIMUM
  median 18 moves, middle half 16–22, the longest 68. 26 of the 100 runs found the shortest possible path (16 moves) and 69 have at most 20 moves. It costs time – every run uses all 300 generations, 5.4 s per run instead of 0.4 s.

SUMMARY

The best configuration solved every run on new seeds, about twice as fast as the default settings. Combined with *--find-best* it finds paths close to the shortest possible one.

DESIGN NOTES

- HALTING AT THE END OF MEMORY is the default because a program that falls off its last cell is simply finished. *--wrap* lets programs loop around instead; the 500-step limit still applies.

- LAST TWO BITS FOR WRT. All four directions are equally likely for a random byte, and a single INC / DEC changes the direction – this makes self-modifying loops that "steer" the hunter easy to evolve.

- STOP AT THE FIRST SOLUTION is what the assignment asks for. That path is usually far from the shortest one (the shortest path on *grid.txt* has 16 moves); *--find-best* keeps evolving and the fitness term *− 0.001 × moves* then shortens the path.

- INSERT AND DELETE MUTATIONS SHIFT EVERY LATER CELL, which breaks jump targets and addresses in that part of the program. They help the program change its structure, but are much more destructive than the per-cell mutations – try *--insert 0 --delete 0* to compare.

- REPRODUCIBILITY. Every random choice uses one seeded generator, so the same seed and settings always give the same run. The seed is printed at the start and stored in the log.